End-to-End MLOps Project: Heart Disease Prediction
(Assignment 01)
Roll No. 2025CT05072 Name: Rupak Mondal

GitHub URL. https://github.com/2025ct05072-Rupak/mlops

Video URL. Recording-20261020_232610.webm

Abstract

This project demonstrates the development and deployment of an end-to-end machine learning solution for heart disease classification using the UCI Heart Disease Dataset. The system incorporates data acquisition, exploratory data analysis (EDA), feature engineering, model development, experiment tracking, model packaging, automated testing, continuous integration and continuous deployment (CI/CD), containerization, Kubernetes deployment, and monitoring.

Logistic Regression and Random Forest are used as candidate classification algorithms. MLflow supports experiment tracking, FastAPI provides the prediction service, Docker packages the application, and Kubernetes manages deployment. Prometheus and Grafana provide operational monitoring and visualization.

The project demonstrates the application of modern MLOps principles to create a reproducible, maintainable, and scalable machine learning workflow.

1. Architecture Overview

The proposed architecture consists of the following stages:

Data Acquisition

Data Preparation

Model Development

Experiment tracking

Automated CI/CD

Model Serving

Containerization

Deployment

Monotoring

End-to-End MLOps Project: Heart Disease Prediction
(Assignment 01)
Roll No. 2025CT05072 Name: Rupak Mondal

Architecture explanation: Data flows from acquisition and preprocessing to model training and evaluation. The selected model is packaged with its preprocessing pipeline and served through an API. CI/CD automates quality checks, Docker provides a portable runtime, Kubernetes manages the deployed service, and monitoring tools track service health and performance.

2. Problem Statement and Dataset

The objective is to develop a binary classification model that predicts the presence or absence of heart disease based on patient health attributes.

Dataset details

- Dataset name: Heart Disease UCI Dataset.
- Source: UCI Machine Learning Repository.
- Dataset acquisition: Python package `ucimlrepo`.
- Commonly used Cleveland subset: 303 records and 13 predictor features.
- Example attributes: age, sex, chest-pain type, resting blood pressure, cholesterol, maximum heart rate, and exercise-induced angina.
- Target: absence or presence of heart disease.

The original target contains values from 0 to 4 in the commonly used version. For binary classification, the target can be transformed into 0 for absence and 1 for presence, with original values greater than zero mapped to presence.

This project is intended for educational purposes and is not a clinically validated diagnostic system.

3. Setup and Installation Instructions

3.1 Prerequisites

The following tools are required:

- Python and pip
- Git and GitHub
- Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn
- MLflow
- FastAPI and Uvicorn
- Pytest and Ruff
- Docker Desktop
- Kubernetes through Docker Desktop or Minikube
- Prometheus and Grafana

End-to-End MLOps Project: Heart Disease Prediction (Assignment 01)

Roll No. 2025CT05072 Name: Rupak Mondal

3.2 Environment setup

Create and activate a Python virtual environment.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

3.3 Dataset acquisition

The dataset can be obtained using the following Python code:

```python
def download_dataset():
    """Download UCI Heart Disease dataset ID 45."""
    DATA_DIR.mkdir(parents=True, exist_ok=True)

    heart_disease = fetch_ucirepo(id=45)
    X = heart_disease.data.features.copy()
    y = heart_disease.data.targets.copy()

    print("Dataset metadata:")
    print(heart_disease.metadata)
    print("\nVariables:")
    print(heart_disease.variables)

    target_col = "num" if "num" in y.columns else y.columns[0]
    original_target = pd.to_numeric(
        y[target_col], errors="coerce"
    )

    df = X.copy()
    df["target"] = (original_target > 0).astype(float)
    df.loc[original_target.isna(), "target"] = float("nan")

    df = df.dropna(subset=["target"]).copy()
    df["target"] = df["target"].astype(int)

    df.to_csv(DATA_PATH, index=False)

    print(f"\nSaved dataset to {DATA_PATH}")
    print("Shape:", df.shape)
    print("\nMissing values:")
    print(df.isna().sum())
    print("\nTarget distribution:")
```

```
print(df["target"].value_counts())

return df
```

4. Data Preparation and Exploratory Data Analysis

EDA is performed to understand the structure, quality, distribution, and relationships within the dataset.

4.1 Data preparation

The following operations are performed:

1. Inspect column names, data types, and dataset dimensions.
2. Identify missing values and unknown markers.
3. Check duplicate records and potentially invalid values.
4. Convert the original target into a binary classification target.
5. Separate predictor features from the target.
6. Split the data into training and testing subsets using stratification.

4.2 EDA visualizations

The following visualizations support data understanding:

Visualization	Purpose
Target distribution	Understand the balance between classes
Feature histograms	Examine distributions and outliers
Correlation heatmap	Identify relationships among numeric variables
Missing-value chart	Identify incomplete features
Feature comparison plots	Compare selected features across target classes

The generated charts should be saved as artifacts and included in the final project report.

4.3 Preprocessing

A Scikit-learn preprocessing pipeline can include:

- `SimpleImputer` for missing values.
- `StandardScaler` for numerical features used by Logistic Regression.
- `OneHotEncoder` for categorical features.
- `ColumnTransformer` to apply transformations to appropriate columns.
- `Pipeline` to combine preprocessing and model training.

Fitting preprocessing inside the pipeline prevents information from the test set leaking into the training process.

5. Feature Engineering and Model Development

Two classification algorithms are considered.

5.1 Logistic Regression

Logistic Regression provides an interpretable baseline for binary classification. It estimates the probability of a record belonging to the positive class.

Potential tuning parameters include regularization strength and class weighting.

5.2 Random Forest

Random Forest combines multiple decision trees and can capture nonlinear relationships and interactions among features.

Potential tuning parameters include the number of trees, maximum depth, minimum leaf size, and class weighting.

5.3 Evaluation strategy

Model performance is assessed using stratified cross-validation and a separate held-out test set.

The evaluation metrics include:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- Confusion matrix

Recall and false-negative counts deserve particular attention because missed positive cases are important in a healthcare-related classification problem.

5.4 Model comparison

Metric	Logistic Regression	Random Forest
Cross-validation accuracy	Measured during training	Measured during training
Test accuracy	Measured on held-out data	Measured on held-out data
Precision	Calculated from predictions	Calculated from predictions

Recall	Calculated from predictions	Calculated from predictions
ROC-AUC	Calculated from probabilities	Calculated from probabilities

The final model should be selected based on cross-validation results, the chosen evaluation criteria, and the trade-off between false positives and false negatives.

6. Experiment Tracking with MLflow

MLflow is used to maintain a record of model experiments and make the development process reproducible.

6.1 Information tracked

Each experiment records:

- Model type and hyperparameters.
- Preprocessing configuration.
- Training and cross-validation settings.
- Evaluation metrics.
- Confusion matrices and ROC curves.
- Fitted model artifacts.
- Relevant data and model version information.

6.2 Running the MLflow interface

```
mlflow ui --host 127.0.0.1 --port 5000
```

Open the following address in a browser:

```
http://127.0.0.1:5000
```

The MLflow interface allows the developer to compare runs, inspect parameters, review metrics, and access saved artifacts.

The selected experiment should be documented with its model name, run identifier, evaluation results, and model artifact location.

7. Model Packaging and Reproducibility

The final model should be saved in a reusable format, such as a Scikit-learn artifact managed through MLflow or a trusted Joblib file.

The saved artifact should contain the complete fitted preprocessing and classification pipeline.

Reproducibility practices include:

- Maintaining a version-controlled code repository.
- Recording dependency versions.
- Fixing random seeds where appropriate.
- Documenting the dataset source and target transformation.
- Recording experiment parameters and model versions.
- Keeping training and inference preprocessing consistent.
- Excluding secrets and sensitive data from version control.

8. Prediction API

FastAPI provides an HTTP interface through which clients can submit health attributes and receive model predictions.

8.1 API endpoints

Endpoint	Purpose
`GET /health`	Check service health
`POST /predict`	Return the binary prediction and positive-class probability
`GET /metrics`	Expose metrics for Prometheus

9. Automated Testing and CI/CD

Automated testing helps verify that data preparation, model training, and API behavior remain reliable when the code changes.

9.1 Testing strategy

Pytest can be used to test:

- Data loading and target conversion.
- Missing-value handling.
- Preprocessing pipeline behavior.
- Model training and prediction outputs.
- API health and prediction endpoints.
- Invalid input handling and metrics exposure.

9.2 CI/CD workflow

GitHub Actions can automate the following steps:

1. Check out the repository.

2. Set up the Python environment.
3. Install the project dependencies.
4. Run linting with Ruff.
5. Execute unit tests using Pytest.
6. Run a model-training smoke test.
7. Verify that expected artifacts are generated.
8. Optionally build the Docker image.
9. Deploy after successful checks, when deployment automation is configured.

9.3 Workflow evidence

The final report can include screenshots showing the GitHub Actions workflow status, test results, training execution, and Docker image build.

10. Docker and Kubernetes Deployment

10.1 Docker containerization

Docker packages the prediction API, its dependencies, and access to the fitted model artifact.

Build the image:

```
docker build -t heart-disease-api:latest .
```

Run the container:

```
docker run --rm -p 8000:8000 heart-disease-api:latest
```

Verify the health endpoint:

```
curl http://localhost:8000/health
```

The application should listen on `0.0.0.0` inside the container so it can receive external requests through the mapped port.

10.2 Kubernetes deployment

Kubernetes manages the API deployment and service lifecycle.

A Deployment defines the application replicas and container configuration. A Service provides network access to the running pods.

Depending on the environment, the API can be exposed using NodePort, LoadBalancer, Ingress, or port forwarding.

10.3 Deployment verification

Deployment verification should confirm that:

- The expected pods are running and ready.
- The Service exposes the correct port.
- Container logs do not show startup failures.
- The API health endpoint responds successfully.
- A prediction request works through the deployed endpoint.

11. Monitoring and Operational Visibility

Prometheus, Grafana, and application logging provide operational visibility into the deployed API.

11.1 Prometheus

Prometheus collects metrics exposed by the API. Useful measurements include request counts, prediction counts, response latency, error counts, and service health.

11.2 Grafana

Grafana connects to Prometheus as a data source and visualizes collected metrics in dashboards.

Recommended panels include:

- API request rate.
- Successful and failed requests.
- Prediction request count.
- Response latency.
- Service health.

11.3 Application logging

Request logs can capture timestamps, routes, HTTP status codes, response duration, and safe error details. Sensitive clinical input values should not be unnecessarily recorded.

The report should include screenshots of the Prometheus target status, a populated Grafana dashboard, and representative application logs.

12. Results and Discussion

The final results section should summarize the measured performance of the candidate models and the behavior of the deployed API.

Discuss:

- Which model performed better under the selected validation strategy.
- The balance between precision and recall.
- The false positives and false negatives shown by the confusion matrix.
- The selected MLflow experiment and saved model artifact.
- Successful API testing and deployment verification.
- The monitoring information collected during test requests.

Results must be reported from actual model runs. No performance values should be assumed in advance.

13. Reproducibility, Security, and Limitations

Reproducibility is supported through version-controlled source code, recorded dependencies, consistent preprocessing, experiment tracking, and saved model artifacts.

Security and operational practices include protecting credentials, validating incoming API requests, avoiding unnecessary sensitive data in logs, and monitoring service errors.

The dataset is relatively small and historically collected. Its evaluation results may not generalize to other populations or clinical settings. The project is an educational demonstration, not a clinically validated diagnostic tool.

A local Kubernetes deployment demonstrates container orchestration but does not by itself establish public-cloud deployment or production readiness.

14. Conclusion

This project demonstrates an end-to-end MLOps workflow for heart disease classification, covering data acquisition, EDA, preprocessing, model development, experiment tracking, model packaging, API development, automated testing, CI/CD, Docker containerization, Kubernetes deployment, and monitoring.

The integration of these components establishes a reproducible development and deployment workflow. Automated tests improve reliability, experiment tracking supports model comparison, and monitoring provides visibility into service behavior.

The project illustrates how machine learning can be integrated into a software delivery lifecycle while emphasizing the importance of data quality, reproducibility, security, and responsible interpretation of model outputs.